

Algorithmique et programmation impérative

Chapitre 1 : Introduction

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 1. Introduction

- 1 Objectifs du cours
- 2 Organisation pratique
- 3 Petite mise au point technique
- 4 Algorithmes et programmes
- 5 Erreurs (bugs)
- 6 Plan du cours
- 7 Glossaire

Objectifs du cours

Objectifs du cours

Etre capable de

- comprendre un **algorithme**
- concevoir un **algorithme** (efficace)
- écrire un **programme**

Organisation pratique

Organisation pratique

- 30 heures de cours
- Transparents : <https://moodle.umons.ac.be>
- 48 heures de travaux pratiques : salles machines (12 séances)
- Examen écrit en janvier (Q1), juin (Q2, uniquement pour B1) et septembre (Q3)
- Test écrit en milieu de Q1

Livres de références

- DOWNEY, A., *Think Python : how to think like a computer scientist*, Green Tea Press (2nd edition, 2015)
www.greenteapress.com/thinkpython2/thinkpython2.pdf
- AHO, A. et ULLMAN, J., *Concepts fondamentaux de l'Informatique*, Dunod (1993)

Organisation pratique

Equipe enseignante

Titulaire : Hadrien MÉLOT

Assistants : Sébastien BONTE, Valentin DUSOLLIER et Guillaume CARDOEN

Bureaux : 2ième étage du De Vinci

Emails : `prenom.nom@umons.ac.be`

Mise en garde concernant les TPs

- L'algorithmique et un langage de programmation, cela s'apprend de manière **active** et avec un travail **personnel** et **régulier**.
- **Avant** d'aller à une séance de TP, bien relire le cours.
- **Erreur fatale** : croire que les TPs vont permettre de comprendre le cours. Les TPs supposent que vous ayez **compris**, pour permettre d'**aller plus loin**.
- Si vous vous sentez perdus, contactez-nous ou pensez à l' **aide à la réussite** (voir horaire).

⇒ voir document “organisation et consignes”

Petite mise au point technique

Quelques notions techniques importantes pour un programmeur

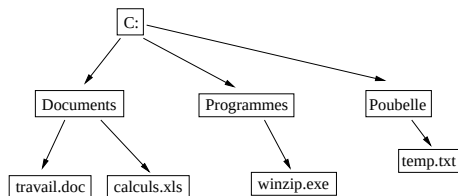
- ❶ Hiérarchie des fichiers (vision graphique et console)
- ❷ Commandes de base dans la console
- ❸ Distinction entre fichiers “texte” et “binaire”

⇒ Ces notions vont être expliquées brièvement ci-après.

Vous vous familiariserez rapidement avec celles-ci lors des Travaux Pratiques.

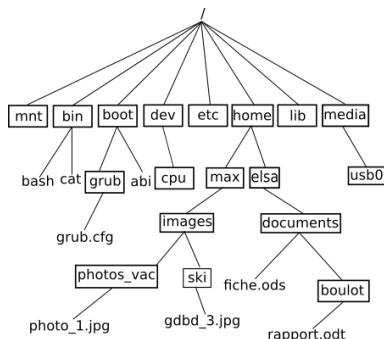
Voir également le tutoriel “ligne de commande” disponible sur moodle.

Hiérarchie des fichiers



Illustré en séance :

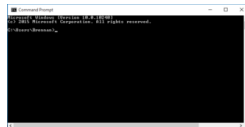
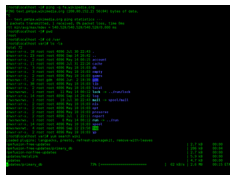
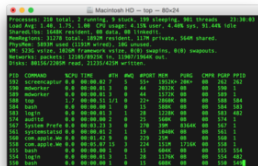
- notion de racine, de répertoire “home” et de répertoire “courant” ;
- vision “graphique” pour voyager dans cette hiérarchie.



Source image de droite : https://pixees.fr/informatiquelycee/n_site/nsi_prem_cmd_base_linux.html

Commandes de base dans la console

Tous les systèmes d'exploitation (OS X, Linux, Unix, Windows, etc.) possèdent une (ou plusieurs) **console(s)** qui permettent de lancer des commandes sans passer par l'interface graphique.



Noms des consoles : terminal (Mac), invite de commande ou command prompt (Windows), shell (Unix), le nom d'un certain type de console (`bash`, `zsh`, etc.), ...

Source des images : <https://fr.wikipedia.org>

Commandes de base dans la console

Pour connaître le répertoire courant (souvent le « home » par défaut quand on lance la console), on utilise la commande

```
pwd
```

sous Linux et Mac ; et la commande

```
cd
```

sous Windows.

Remarque : la commande `cd` sans argument a un autre comportement sous Linux et Mac, voir ci-après.

Exemple :

```
hmelot@Opeth ~ % pwd  
/Users/hmelot
```

Commandes de base dans la console

Pour lister les fichiers et les répertoires contenus dans le répertoire courant, on utilise la commande

`ls`

sous Linux et Mac ; et la commande

`dir`

sous Windows.

Commandes de base dans la console

Pour changer de répertoire, on utilise la commande

```
cd NomRepertoire
```

ou

```
cd Chemin/Vers/Repertoire
```

Cas spéciaux :

- « cd .. » pour « remonter » d'un répertoire
- « cd » pour se placer dans le « Home » ; « cd / » pour la racine (Linux et Mac uniquement)

⇒ illustration de la vision “console”
pour voyager dans la hiérarchie des fichiers

Distinction entre fichiers “texte” et “binaire”

Un fichier de **texte** (ou fichier texte brut) ne contient que des caractères (il utilise pour cela une forme de codage comme le code ASCII).

Un fichier **binaire** est un fichier qui n'est pas interprétable directement sous forme de texte. Il utilise un codage binaire pour stocker ses données (il peut cependant y avoir aussi des caractères dans ce type de fichiers). Par exemple, un fichier `.mp3` est un fichier binaire stockant des informations pour représenter un morceau de musique, un fichier `.jpg` pour représenter une image. Il faut utiliser un programme spécifique, qui connaît le format, pour décoder ce type de fichiers.

Distinction entre fichiers “texte” et “binaire”

Il existe des “**éditeurs** de texte” (par ex., gedit, notepad) pour les fichiers “texte”.

Question : d’après vous, un fichier Word ou OpenOffice est un fichier texte ou binaire ?

⇒ illustration via la commande `more`
(fonctionne sous Linux, Mac et Windows)

Distinction entre fichiers “texte” et “binaire”

Il existe des “**éditeurs** de texte” (par ex., gedit, notepad) pour les fichiers “texte”.

Question : d’après vous, une fichier Word ou OpenOffice est un fichier texte ou binaire ?

Ce sont des fichiers binaires ! Pour les coder et les décoder, on doit utiliser un programme spécifique (programmes de **traitement** de texte). Ils ne contiennent pas que du texte, ils codent également toute une série d’informations relatives au **format** et à la manière de **présenter** le document, notamment.

⇒ **illustration via la commande** `more`
(fonctionne sous Linux, Mac et Windows)

Distinction entre fichiers “texte” et “binaire”

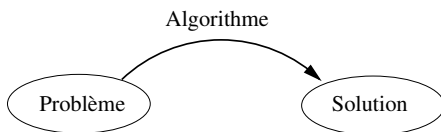
On écrit du code informatique dans des fichiers texte. Certains éditeurs de texte reconnaissent (via l’extension du fichier) qu’il s’agit de code et ajoutent une **coloration syntaxique** pratique pour le programmeur. Mais en réalité, le fichier de code est bien un fichier de texte brut.

Illustration : création d’un fichier `hello.py` qui contient une ligne de code en Python et comparaison des vues via la commande `more` et via un éditeur de code.

Algorithmes et programmes

Qu'est-ce qu'un algorithme ?

Tâche principale de l'informaticien : résoudre des problèmes



- ❶ spécification du problème
 - ▶ qu'a-t-on besoin pour résoudre le problème (entrées) ?
 - ▶ que doit-on fournir comme solution (sorties) ?
- ❷ résolution : comment trouver la solution (méthode, algorithme) ?

Définition

Un **algorithme** est une séquence d'étapes précises et non ambiguës pouvant être exécutées de façon automatique.

Exemples d'algorithmes

Spécification du problème :

Entrée : L'heure h (entier allant de 0 à 23)

Sortie : Un message de bienvenue spécifique

Algorithme :

- 1: **si** $h \leq 6$ **alors**
- 2: dire "Bonne nuit"
- 3: **sinon si** $h \leq 18$ **alors**
- 4: dire "Bonjour"
- 5: **sinon**
- 6: dire "Bonsoir"
- 7: **fin si**

Exemples d'algorithmes

Spécification du problème :

Entrée : Un dictionnaire et un mot

Sortie : La définition du mot recherché

Algorithme :

- 1: lire le premier mot du dictionnaire
- 2: **tant que** le mot lu n'est pas le mot recherché **faire**
- 3: lire le mot suivant
- 4: **fin tant que**
- 5: lire la définition du mot recherché

Exemples d'algorithmes

De manière plus efficace :

- 1: ouvrir le dictionnaire au milieu
- 2: **tant que** la page courante ne contient pas le mot **faire**
- 3: ouvrir le dictionnaire au milieu de la partie restante
- 4: **fin tant que**
- 5: **rechercher** la définition du mot dans la page courante

Exemples d'algorithmes

De manière plus précise et moins ambiguë :

```
1: pageDebut  $\leftarrow$  1
2: pageFin  $\leftarrow$  numéro de la dernière page du dictionnaire
3: pageCourante  $\leftarrow$  pageDebut + (pageFin - pageDebut)/2
4: tant que pageCourante ne contient pas le mot faire
5:   si le mot se trouve avant pageCourante alors
6:     pageFin  $\leftarrow$  pageCourante - 1
7:   sinon
8:     pageDebut  $\leftarrow$  pageCourante + 1
9:   fin si
10:  pageCourante  $\leftarrow$  pageDebut + (pageFin - pageDebut)/2
11: fin tant que
12: rechercher la définition du mot dans pageCourante
```

Qu'est-ce qu'un programme ?

Définition

Un **programme** est une séquence d'instructions qui spécifie comment réaliser un calcul ou une tâche. Ils sont décrits à l'aide de **langages de programmation**.

Remarque : le mot “programme” est utilisé aussi pour l'application qui “tourne” sur la machine. Beaucoup de langages différents mais les instructions permettent par exemple de :

- réaliser des opérations mathématiques (par ex. addition, multiplication)
- obtenir des données au clavier, depuis un fichier, etc.
- afficher des données à l'écran ou écrire des données dans un fichier
- exécution conditionnelle : vérifier si certaines conditions sont respectées et exécuter la séquence d'instructions appropriée
- réaliser une tâche de manière répétitive

Langages de programmation

On distingue les langages

- de **haut niveau** : compréhensible par l'humain (souvent langage formel avec des mots issus de l'anglais) : Python, Java, C, C++, PHP, Scheme, Pascal, etc.
- de **bas niveau** : exécutables par un ordinateur (langages machine ou assembleur)

Avantages d'un programme écrit dans un langage de haut niveau :

- beaucoup plus facile à écrire et à (re)lire ;
- plus rapide à écrire ;
- plus court ;
- **portable**
 - ▶ peut être utilisé sur différents types d'ordinateurs avec peu ou pas de modifications ;
 - ▶ un programme de bas niveau n'est exécutable que sur un type bien particulier d'ordinateurs et doit être réécrit pour d'autres.

Langages de programmation

Remarque

La grande majorité des programmes sont écrits en langages de haut niveau (sauf quelques applications spécialisées)

En pratique

Un programme compréhensible par l'humain (haut niveau) est traduit en un programme compréhensible par la machine (bas niveau) de manière automatique (par un programme dédié à cette tâche)

2 types de programmes “traducteurs” :

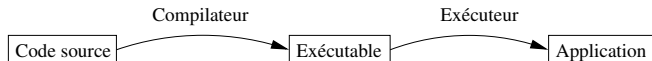
- compilateurs
- interpréteurs

Compilateurs

Définition

Un **compilateur** est un programme qui lit un **code source** et le traduit en un **exécutable**, **avant** l'exécution du programme.

- **code source** : programme écrit dans un langage de haut niveau spécifique
- **exécutable** : code compilé qui peut être exécuté sans être traduit à nouveau
- **application** : exécution du programme (qui “tourne” sur la machine)



Compilateurs

Exemple

Programme écrit en C, compilé avec `gcc` et exécuté dans une console.

```
#include <stdio.h>
#include <time.h>

int main() {
    time_t timestamp = time(NULL);
    struct tm * t = localtime(&timestamp);
    int hour = t->tm_hour;

    if (hour <= 6)
        printf("Bonne nuit. ");
    else if (hour <= 18)
        printf("Bonjour. ");
    else
        printf("Bonsoir");

    printf("Il est %02uh %02umin %02usec.\n", t->tm_hour, t->tm_min, t->tm_sec);

    return 0;
}
```

Compilateurs

Exemple

Programme écrit en Java, compilé avec `javac` et exécuté dans une console (avec `java`).

```
import java.util.Calendar;

public class Greeter {
    private Calendar d;

    public Greeter() {
        updateTime();
    }

    public void updateTime() {
        d = Calendar.getInstance();
    }
}
```

Compilateurs

```
public void sayHello() {
    updateTime();
    int hour = d.get(Calendar.HOUR_OF_DAY);

    if (hour <= 6)
        System.out.print("Bonne nuit. ");
    else if (hour <= 18)
        System.out.print("Bonjour. ");
    else
        System.out.print("Bonsoir. ");

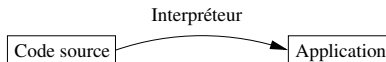
    System.out.println("Il est " + hour + "h " + d.get(Calendar.MINUTE)
        + "min " + d.get(Calendar.SECOND) + "sec");
}

public static void main(String[] args) {
    Greeter g = new Greeter();
    g.sayHello();
}
```


Interpréteurs

Définition

Un **interpréteur** est un programme qui lit un **code source** et l'exécute pas à pas.



Exemple

Programme interprété par Python des 2 façons suivantes :

- **mode script** (à partir d'un fichier)
- **mode interactif** (dans une console ou dans **IDLE**)

Python : mode script

Fichier say_hello.py

```
from datetime import datetime

d = datetime.now()
hour = d.hour

if hour <= 6:
    print('Bonne nuit. ', end='')
elif hour <= 18:
    print('Bonjour. ', end='')
else:
    print('Bonsoir. ', end='')

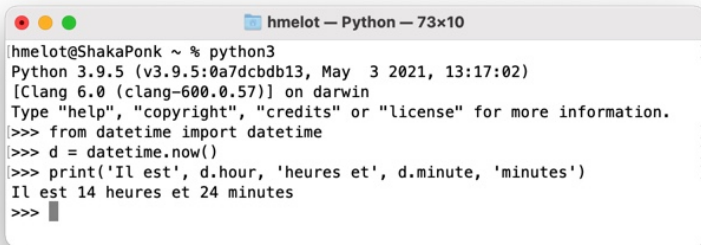
print('Il est', d.hour, 'h', d.minute, 'min', d.second, 'sec')
```

A lancer avec la commande `python3 say_hello.py`

Bonjour. Il est 10 h 42 min 13 sec

Python : mode interactif

- Le mode interactif se lance en tapant simplement `python3` dans la console
- Les caractères `>>>` indiquent que l'interpréteur est prêt à recevoir des instructions (`prompt` ou `invite de commande`)



```
hmelot@ShakaPonk ~ % python3
Python 3.9.5 (v3.9.5:0a7dcdbd13, May  3 2021, 13:17:02)
[Clang 6.0 (clang-600.0.57)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> from datetime import datetime
>>> d = datetime.now()
>>> print('Il est', d.hour, 'heures et', d.minute, 'minutes')
Il est 14 heures et 24 minutes
>>> █
```

Le mode interactif est pratique pour tester des petites choses, pas pour écrire un programme élaboré.

Choix du langage pour le cours

- Le langage utilisé dans ce cours pour implémenter les algorithmes sera le Python (version 3)
- Attention : la version 2 est incompatible !
- Sur moodle : différences principales entre Python 2 et Python 3 (si vous consultez des examens antérieurs à septembre 2016)
Exemple : en Python 2, il n'est pas nécessaire d'utiliser des parenthèses avec `print`.
- Si possible : installez Python 3 chez vous au plus vite
- <https://www.python.org> (téléchargement, documentation, etc.)

Pourquoi Python ?

Principalement car il est un langage **simple** au niveau de la syntaxe, et donc bien adapté d'un point de vue **pédagogique**.

Ce n'est pas seulement un langage pédagogique. Il est utilisé par beaucoup d'entreprises dont Yahoo, Google, Walt Disney, la NASA, etc.

<https://wiki.python.org/moin/OrganizationsUsingPython>

Programmation impérative ?

Il existe plusieurs *paradigmes de programmation*. Un paradigme

- est une approche pour structurer et organiser le code
- définit une manière particulière de penser et de résoudre les problèmes
- définit des concepts et des abstractions qui guident le processus de développement

Exemples : programmation impérative, programmation orientée objets, programmation fonctionnelle, programmation logique, ...

Programmation impérative ?

Dans ce cours, nous nous concentrerons sur un des paradigmes permis par Python.

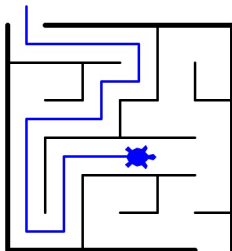
La programmation impérative

- se concentre sur la séquence d'instructions qui modifient l'état du programme
- spécifie **comment** atteindre un objectif en donnant une série d'étapes ou d'ordres à exécuter

Remarque : nous verrons cependant, dans le dernier chapitre, une introduction à la programmation orientée objets (voir également le cours de B. Quoitin).

Illustration

Pour illustrer visuellement d'autres algorithmes, et rencontrer des tortues qui nous aideront lors de ce cours, je vous propose de voir l'exécution de différents scripts Python.



Erreurs (bugs)

Erreurs (bugs)

Les erreurs sont fréquentes quand on programme.

Trois sortes d'erreurs :

- erreurs de **syntaxe**
- erreurs à l'exécution (ou **exceptions**)
- erreurs **sémantiques** (ou logiques)

Deboguer (debug) : processus de repérage et correction des erreurs

Exemple de programme sans erreur :

```
>>> print('La somme des deux premiers nombres entiers positifs est', (1+2))
```

```
La somme des deux premiers nombres entiers positifs est 3
```

Erreurs de syntaxe

Définition

Une **erreur de syntaxe** est une violation des règles du langage de programmation. Celle-ci est signalée par le compilateur ou l'interpréteur lors de son analyse (**parsing**).

Exemple (manque une parenthèse) :

```
>>> print('La somme des deux premiers nombres entiers positifs est', 1+2))
```

```
SyntaxError: invalid syntax
```

Exemple (manque une apostrophe) :

```
>>> print('La somme des deux premiers nombres entiers positifs est, (1+2))
```

```
SyntaxError: EOL while scanning string literal
```

Exceptions

Définition

Une **exception** (ou erreur à l'exécution) est une erreur qui apparaît après le lancement du programme quand quelque chose d'exceptionnel se produit et qui n'est pas due à une violation de la syntaxe.

Exemples : ouverture d'un fichier sur lequel l'utilisateur n'a pas les droits d'accès, perte de la connexion réseau, etc.

Erreurs sémantiques

Définition

Une **erreur sémantique** (ou erreur logique) se produit quand le programme ne donne pas le résultat attendu.

- apparaît quand le programme est syntaxiquement correct mais la logique des instructions est erronée
- peut être difficile à identifier et corriger (travail de “détective” : savoir déboguer est une importante qualité du programmeur)

Exemples :

```
>>> print('La somme des deux premiers nombres entiers positifs est', (1+3))
```

La somme des deux premiers nombres entiers positifs est 4

```
>>> print('La somme des trois premiers nombres entiers positifs est', (1+2))
```

La somme des trois premiers nombres entiers positifs est 3

Plan du cours

Plan du cours

- 1 Introduction
- 2 Types, variables et expressions
- 3 Fonctions
- 4 Instructions conditionnelles
- 5 Récursivité
- 6 Itérations et chaînes de caractères
- 7 Listes
- 8 Prouver les propriétés des algorithmes
- 9 Complexité : évaluer l'efficacité des algorithmes
- 10 Fichiers
- 11 Exceptions
- 12 Introduction aux objets

Glossaire

Glossaire

résolution de problème : processus de formulation d'un problème (spécification), trouver une solution et exprimer la solution

algorithme : séquence d'étapes précises et non ambiguës pouvant être exécutées de façon automatique

programme : séquence d'instructions écrites dans un langage de programmation particulier et qui spécifie comment réaliser un calcul ou une tâche

langage de haut niveau : un langage comme Python facile à lire et écrire pour l'humain

langage de bas niveau : un langage exécutable par un ordinateur (aussi appelé langage machine ou assembleur)

portabilité : le fait qu'un programme puisse être exécuté sur plus d'une sorte d'ordinateurs

interpréter : exécuter un programme écrit dans un langage de haut niveau en le traduisant pas à pas

compiler : traduire un programme écrit dans un langage de haut niveau en un langage de bas niveau en une fois, en vue de sa future exécution

code source : un programme écrit en langage de haut niveau avant sa compilation ou son interprétation

exécutable : un programme après compilation, prêt à être exécuté

prompt : les caractères >>> qui indiquent que l'interpréteur est prêt à recevoir des instructions

Glossaire

script : un programme stocké dans un fichier (en vue d'être interprété)

mode interactif : une manière d'utiliser Python en tapant les instructions via le prompt

mode script : une manière d'utiliser Python en lisant les instructions depuis un fichier

bug : une erreur dans un programme

deboguer : le processus d'identification et de correction des 3 types d'erreurs de programmation

syntaxe : la structure et les règles d'un langage de programmation

erreur de syntaxe : une violation des règles d'écriture dans un programme qui le rend impossible à interpréter ou à compiler

exception : une erreur détectée pendant l'exécution du programme

sémantique : la signification (la logique, le sens) d'un programme

erreur sémantique : une erreur dans un programme qui fait que quelque chose de différent de ce que le programmeur voulait réaliser se produit

parser : examiner un programme et analyser sa structure syntaxique

print : fonction (voir ch. 3) utilisée pour afficher une valeur à l'écran